

Autonomous LiDAR-Based Mobile Robot

Technical Report



Completed physical prototype

Author: Ilya Mozhaev

Project period: 2024–2025

Original project: School No. 1502, Moscow

Moscow, 2025

Contents

1. Project Motivation and Objective.....	3
2. Mobile Platform Selection.....	3
3. Hardware Architecture	4
3.1. On-board computer.....	4
3.2. Motors and wheel encoders	4
3.3. LiDAR	4
3.4. Motor control and power	4
4. Mechanical Integration	5
5. Software and ROS Integration.....	5
6. Mapping and Navigation	6
7. Project Results	6
8. Scope and Limitations	7
9. Components and Software Mentioned in the Original Report	7

1. Project Motivation and Objective

The project investigated an autonomous robotic platform intended as a prototype for field and lawn-care applications. The motivating problem was that a mobile lawn-care robot should cover an area uniformly while reducing skipped regions and unnecessary repeated passes.

The stated objective of the original project was to create a prototype unmanned system for field processing. The project tasks were to build the robot prototype, apply ROS, create a virtual model, integrate LiDAR and other sensors, and develop and test movement and mapping algorithms.

The project therefore combined mechanical construction, electronics, sensing, Linux/ROS software, odometry, and autonomous navigation in a single physical prototype.

2. Mobile Platform Selection

Several mobile-drive configurations were considered: a two-wheel differential drive with passive supports, skid-steer drive, tracked differential drive, and Ackermann steering. The two-wheel differential configuration was selected because it is mechanically simple, comparatively inexpensive, straightforward to control, and permits wheel odometry using two rotation sensors.

Drive type	Relevant characteristics from the original study
Two-wheel differential	Simple construction and control; wheel odometry requires only two rotation sensors; suitable mainly for even surfaces.
Skid-steer	Better terrain capability, but requires more wheel sensors and provides less accurate odometry than a two-wheel platform.
Tracked differential	High traction, but more complex mechanics and odometry due to track slippage.
Ackermann steering	Suitable for faster robots on even surfaces, but requires additional steering mechanisms.

3. Hardware Architecture

3.1. On-board computer

A Raspberry Pi 4 Model B with 4 GB of RAM was selected as the on-board computer. It was selected because of its compact size, modest power requirements, low cost, extensive documentation, and the availability of the functionality required for the project.

3.2. Motors and wheel encoders

The platform used TT geared DC motors with integrated quadrature magnetic encoders. The motors were specified as 160 rpm, 6 V, with a 120:1 gearbox. The encoder resolution was 16 PPR on the motor shaft; with the gearbox, the original report calculated 1920 pulses per wheel revolution.

Encoder channels A and B were read using Raspberry Pi GPIO hardware interrupts so that rapid signal changes would not be missed. The encoders were used as the basis for wheel odometry.

3.3. LiDAR

The project used an RPLIDAR A1 from SLAMTEC. The original report specifies a 360-degree scan, an update frequency of approximately 2–10 Hz, and a range of about 8 m. It was connected to the Raspberry Pi through USB using the supplied USB-UART interface.

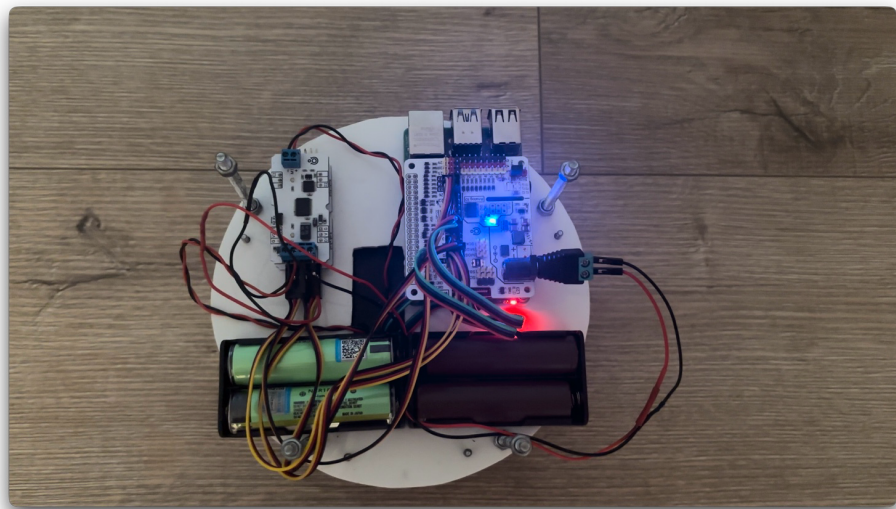
3.4. Motor control and power

A two-channel H-bridge Troyka module was used to control the two DC motors, with a stated maximum current of 1.2 A per channel. The robot used Li-ion batteries; the original report lists HUNGLIANG 18650 cells rated at 4.2 V and 8000 mAh. A Troyka-HAT expansion board was used to simplify connection of multiple devices to the Raspberry Pi GPIO.

4. Mechanical Integration

The project included custom mechanical parts produced by 3D printing. A disk-shaped mounting panel was designed and printed to hold the Raspberry Pi and other electronic components. The Raspberry Pi was fixed to the panel using M2.5 hardware, while the battery holders were attached with M2 hardware.

A separate mounting platform was designed and 3D-printed for the LiDAR. The RPLIDAR A1 was fixed to this platform with M2.5 hardware, and the platform was installed on the robot using M3 standoffs.



Prototype electronics mounted on the mobile platform

5. Software and ROS Integration

ROS Noetic was installed on both the Raspberry Pi and a desktop computer running Ubuntu 20.04.1 LTS. The two machines were connected to the same Wi-Fi network and configured to communicate through ROS.

The original project documentation lists the following ROS packages: `Abot_base` for the hardware interface, `Abot_control` for robot controllers, `Abot_description` for the robot description, and `Abot_teleop` for remote control.

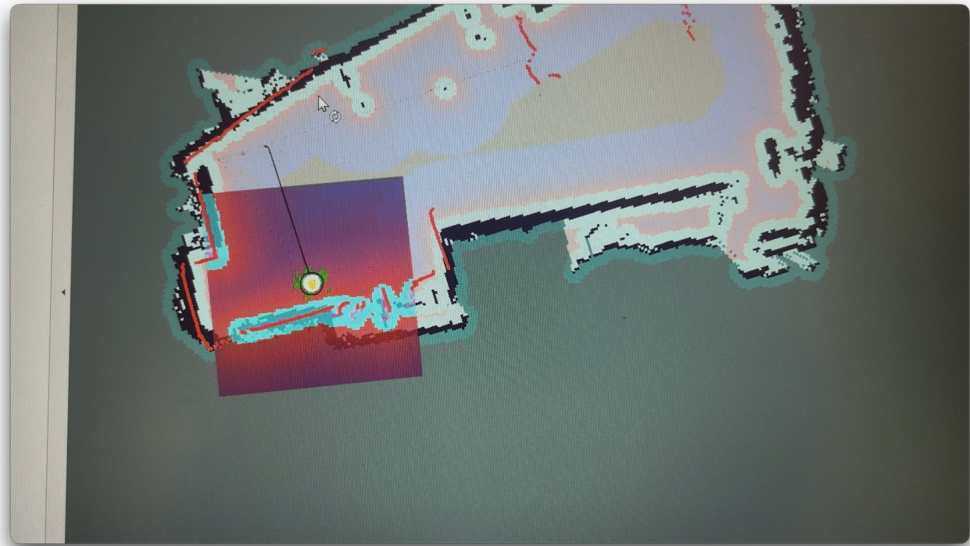
Drivers used to connect the Raspberry Pi to the wheel encoders were added to the project packages and written in C++. Sensor descriptions were organized using Xacro files. The LiDAR description was placed in `abot_sensors.xacro` and included from the main `abot.xacro` robot description.

For the RPLIDAR A1, the project used the rplidar_ros ROS package. A Linux device alias was configured so the software could consistently identify the USB device corresponding to the LiDAR.

6. Mapping and Navigation

LiDAR measurements and wheel odometry provided the information required for the robot to operate without external orientation infrastructure. The original project report states that movement and mapping algorithms were created and tested, and that the resulting prototype could orient itself in space and move along an economical trajectory.

The surviving original report does not contain enough implementation detail to separate, with confidence, which navigation and planning functions were implemented directly in project code and which were provided by existing ROS packages. For that reason, this report does not claim a from-scratch implementation of the complete navigation stack.



Mapping and navigation visualization captured during project testing

7. Project Results

The completed work produced and demonstrated a functioning experimental robot prototype and navigation software. ROS was applied to the physical platform, a

virtual model was created, LiDAR and other sensors were integrated, and movement and mapping functionality was tested.

The final prototype was intended to navigate without external orientation devices. The project focused on obtaining a more systematic and economical motion pattern for area coverage rather than relying on unsystematic movement.

8. Scope and Limitations

This document presents a concise English-language version of the original 2025 project report. It preserves the main technical content of the original work, including the design of the physical prototype, hardware selection and integration, ROS-based software architecture, sensor integration, mapping, and navigation. Theoretical discussions, background material, and detailed comparisons of alternative hardware and design approaches from the original Russian-language report have been omitted or condensed to focus on the practical development and results of the project.

9. Components and Software Mentioned in the Original Report

Category	Component / software
Computer	Raspberry Pi 4 Model B, 4 GB RAM
Drive	Two-wheel differential drive with passive supports
Motors	TT geared DC motors, 160 rpm, 6 V, 120:1
Encoders	Quadrature magnetic encoders, 16 PPR
Range sensor	RPLIDAR A1
Motor driver	Two-channel Troyka H-bridge, up to 1.2 A/channel
GPIO expansion	Troyka-HAT
Power	Li-ion 18650 cells (specified in the original report as 4.2 V, 8000 mAh)
Operating systems	Linux; Ubuntu 20.04.1 LTS on the desktop computer
Robotics framework	ROS Noetic
ROS packages	Abot_base, Abot_control, Abot_description, Abot_teleop, rplidar_ros
Robot description	Xacro / URDF-related project files
Programming	C++ mentioned for encoder drivers